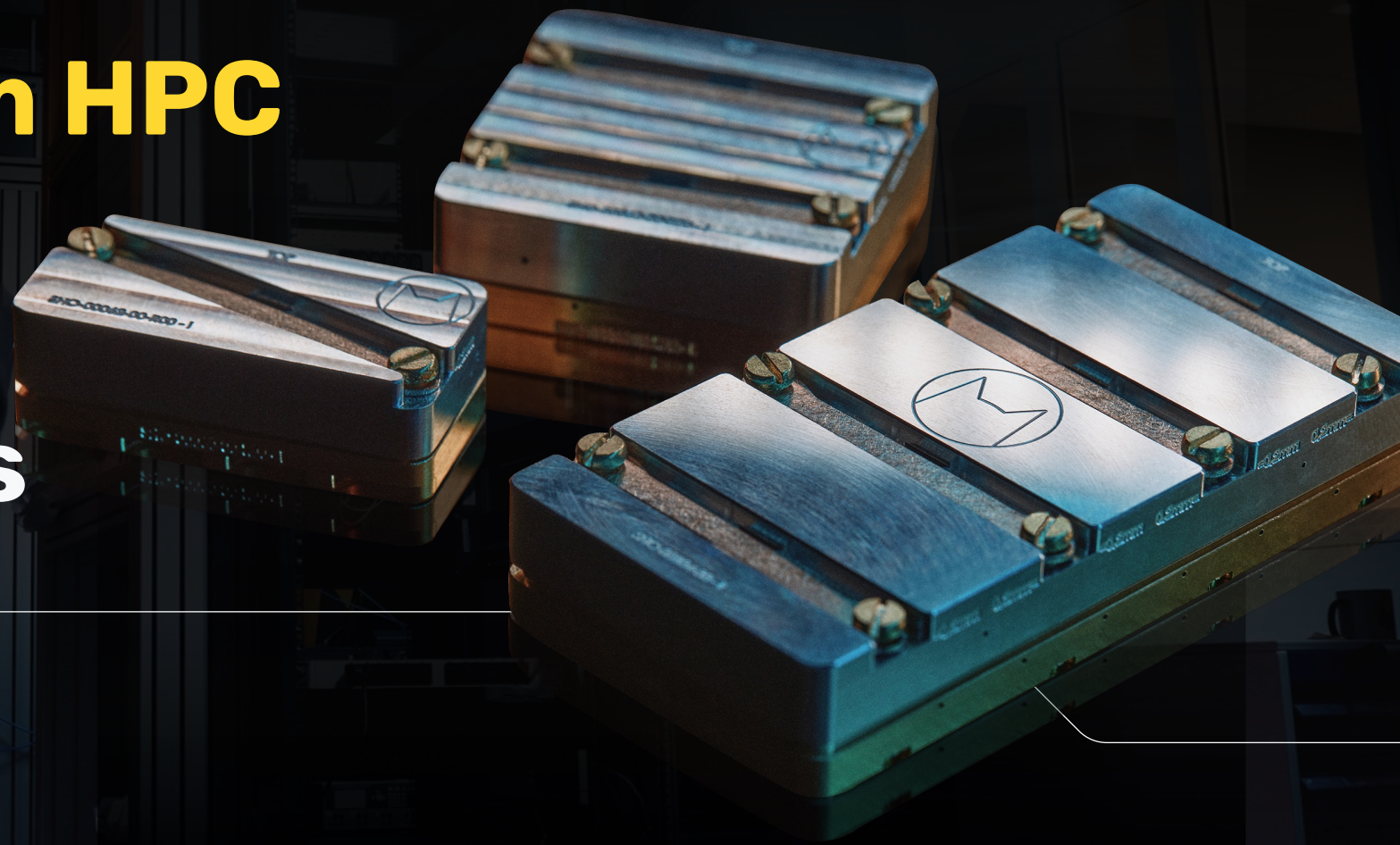
ALICE & BOB

Hybrid Quantum HPC

Stratifications and Hypotheses



Kevin D. Kissell
March 30, 2026

Who are Alice & Bob
...and where am I coming from with all this?

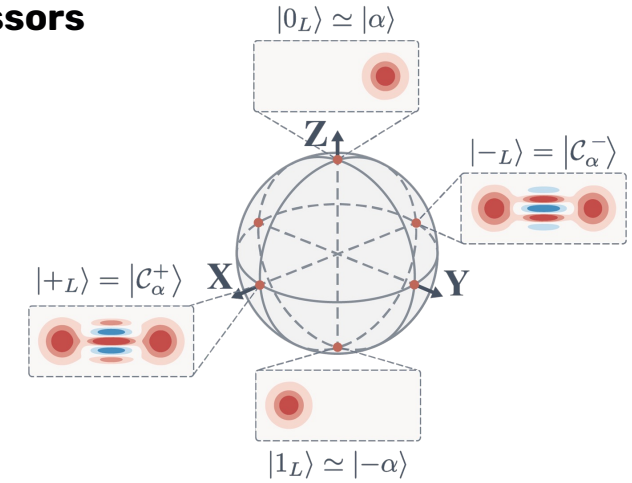
Alice & Bob are building quantum processors based on dissipative cat qubits

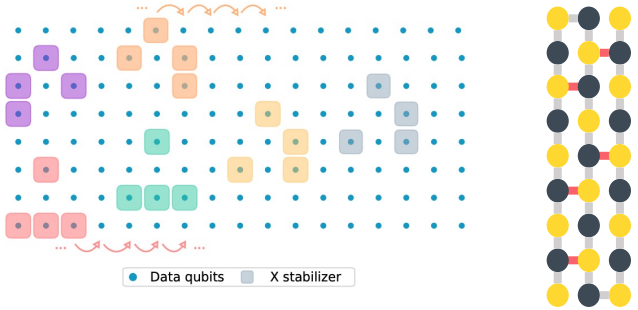
Self-stabilization against bit-flip errors reduces error correction overhead by 1-2 orders of magnitude.

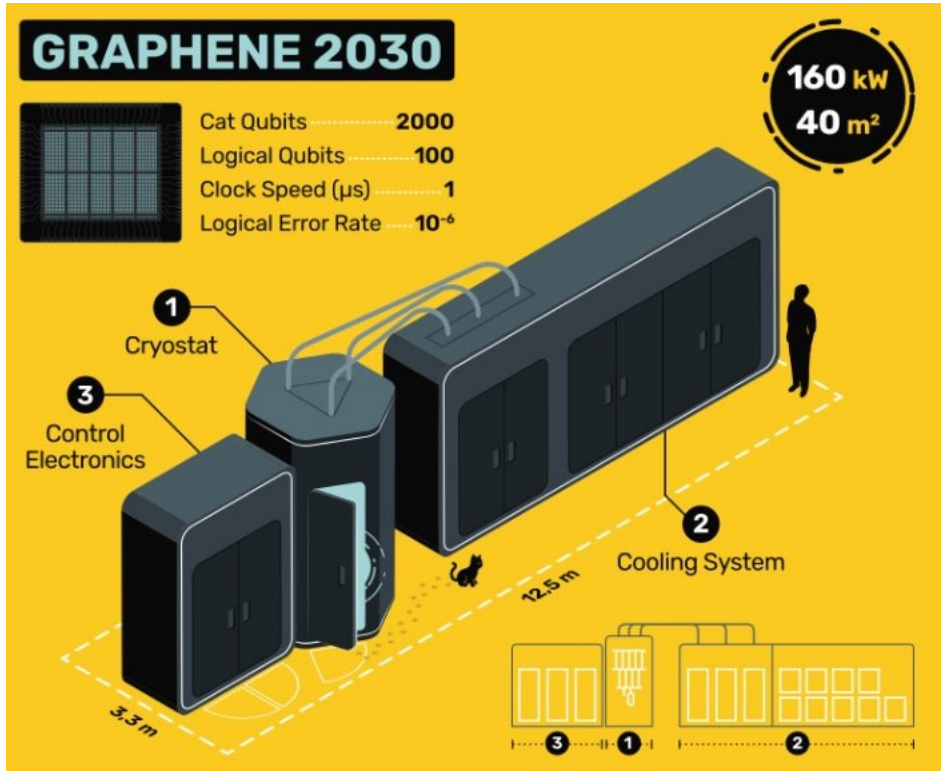
Superconducting semiconductor modality means high density, fast operations, and ready tooling for 2.5D packaging.

We should thus be able to build a fault-tolerant quantum computer capable of running "beyond classical" physics simulations in a single cryostat.

Distinct physical and logical gate sets. Not at all optimised for NISQ.
Targeting integration of FTQC with produ scientific computing models.







Observations about Hybrid Quantum Processing

- Main programs will remain classical
- Quantum information and operations dealt with strictly by reference
- Quantum gates support predicated execution, but not control flow
- Quantum state is expensive to maintain
- Expression of high-order quantum information very expensive classically
- Expression of large volumes of high-precision data very expensive in quantum

How would we integrate such a processor with HPC, from first principles?

A Hierarchy of Quantum Computing Abstractions

Reasoning about the architecture of hybrid quantum HPC computers

Classical Management of Quantum Resources
No awareness of quantum information flow

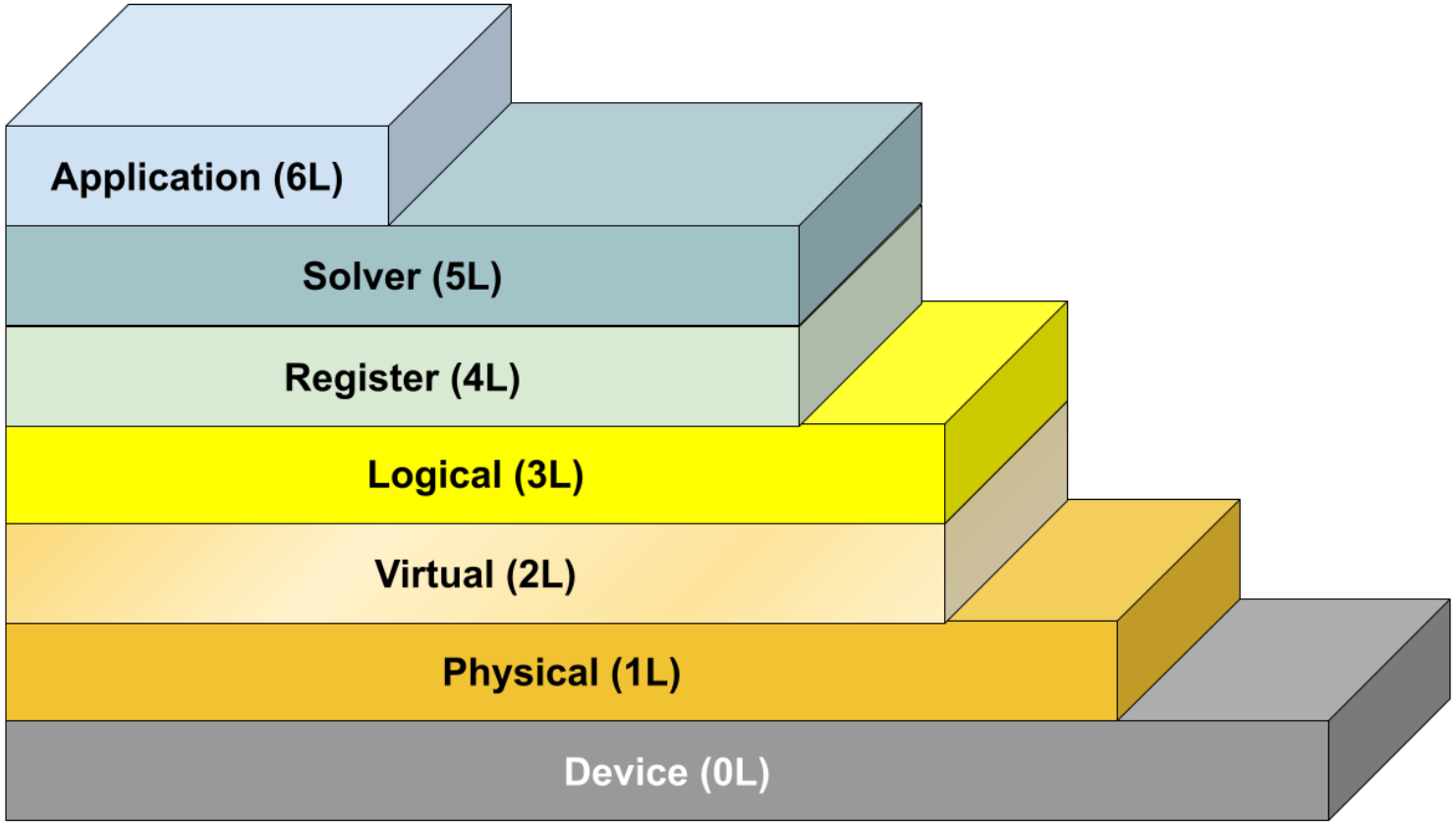
Classical Invocation of Quantum Solvers
Classical management of quantum information flow

Hybrid Implementation of Quantum Solvers
Logical quantum operations + classical control

Expression of fault-tolerant quantum algorithms
The quantum computing primitives

Hybrid Implementation of Logical Quantum operations
Physical quantum operations* + classical control

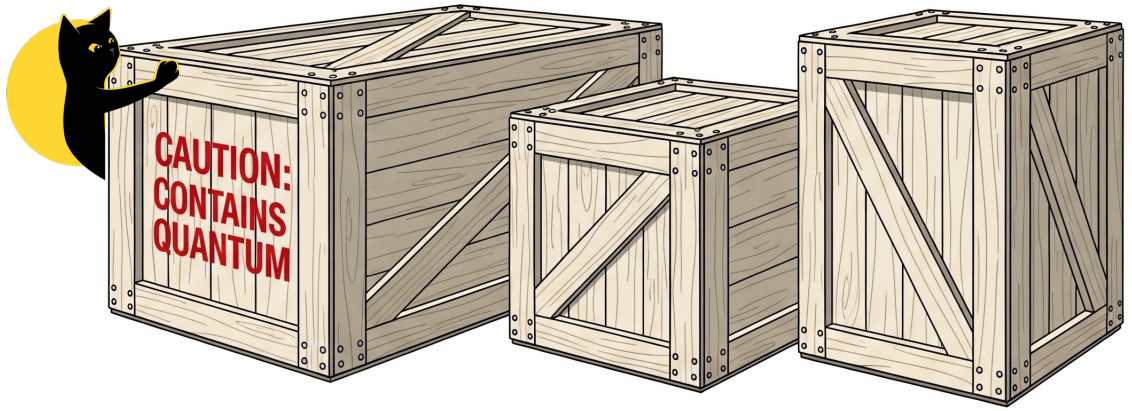
Expression of Physical quantum algorithms
Based on controllable quantum system of device



An Application Layer

Classical Management of Quantum Resources

- Quantum applications are qualitatively different, and must be handled as such, regardless of modality. **OS needs to reason about resources without knowledge of algorithms.**
- Quantum jobs cannot be pre-empted and rescheduled
- Cannot copy quantum information for checkpoint/restart
- Quantum resource estimation needed for efficiency
- Classical components can be run in distinct processes that may or may not be co-located with quantum processing. Decoupling quantum and classical processing decouples resources



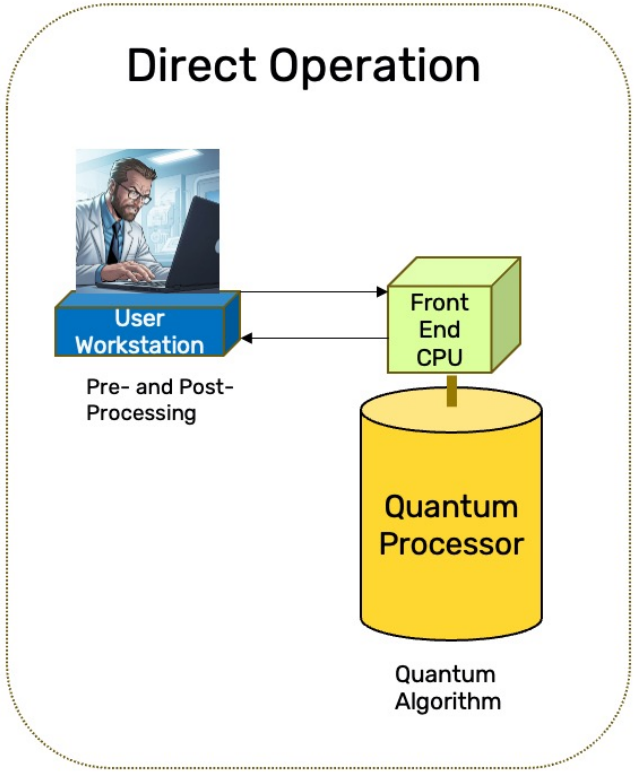
Automating Scaling of Hybrid Quantum Applications

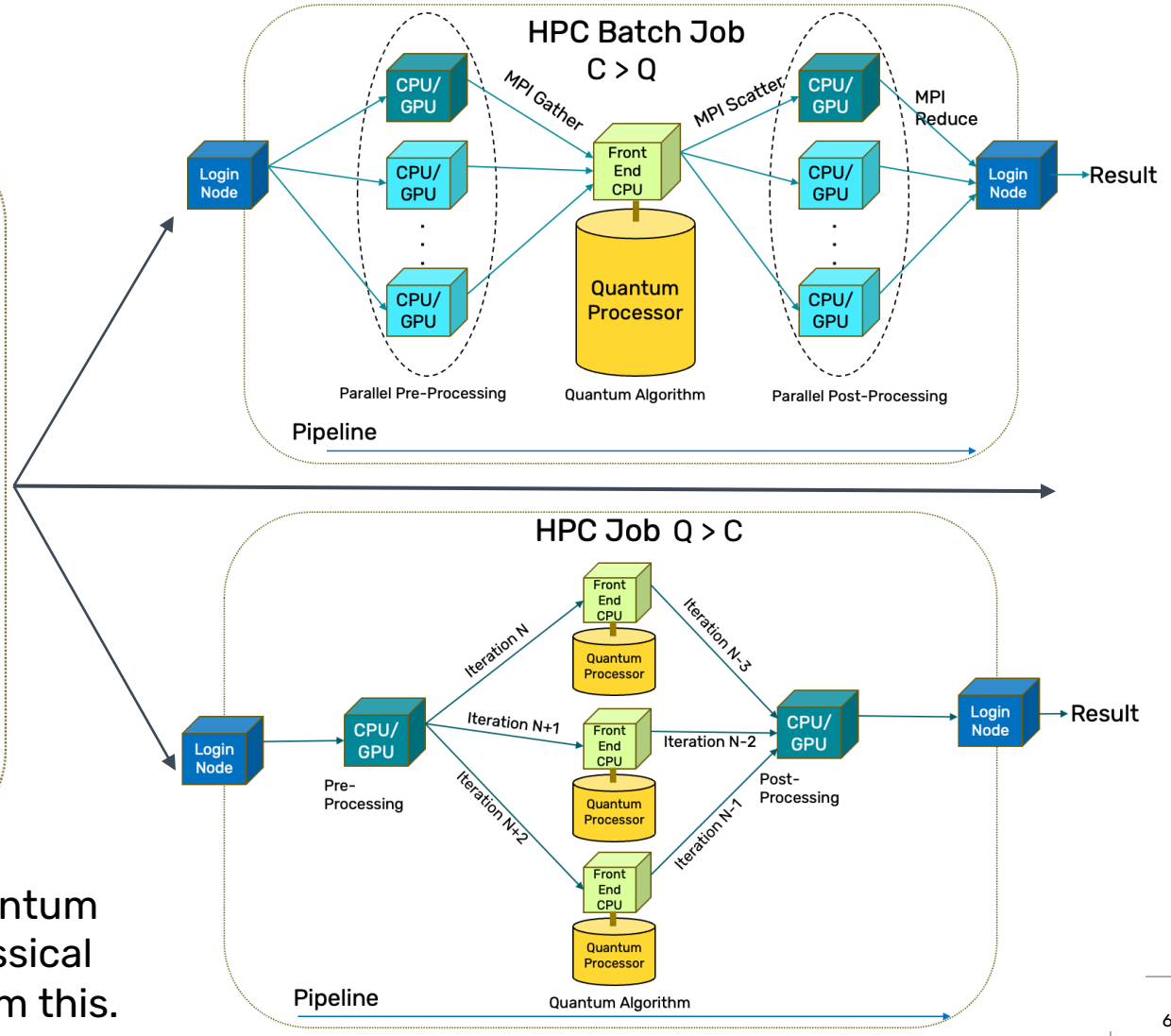
Reasoning about the architecture of hybrid quantum HPC computers

Moving from algorithmic experimentation to production simulation:

Applications can operate as pipelines, with multiple phases or iterations running in parallel.

Parallelism of either quantum or classical phases can be scaled to balance occupation of CPUs, QPUs, to the extent possible.





The first instruction of a quantum HPC application will be a classical instruction. A lot follows from this.

Decentralized Control

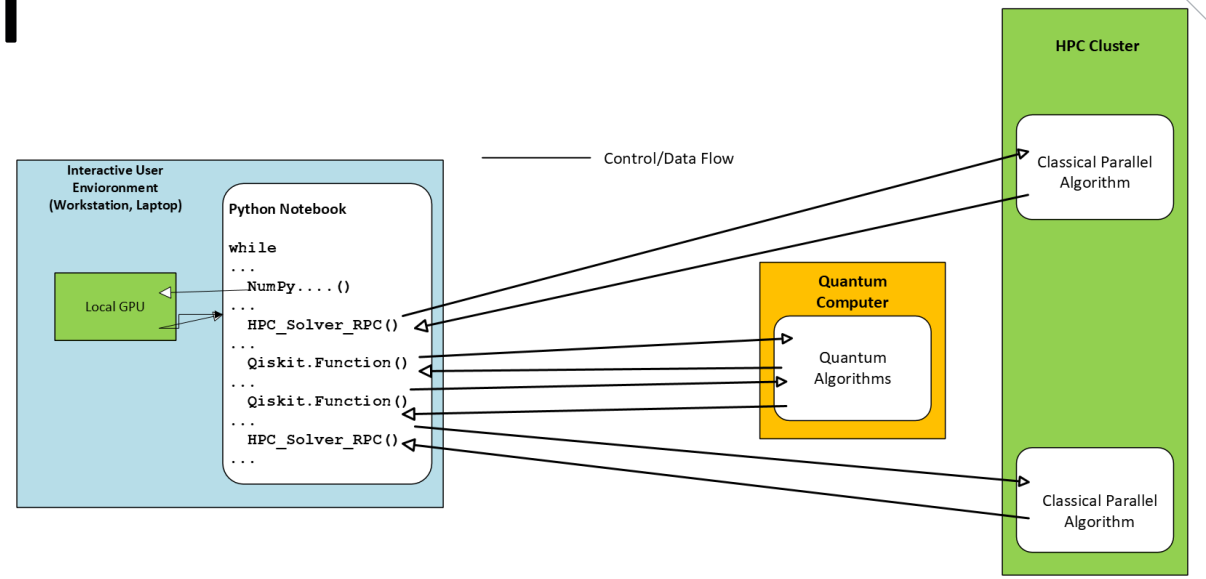
Moving away from a “Script”

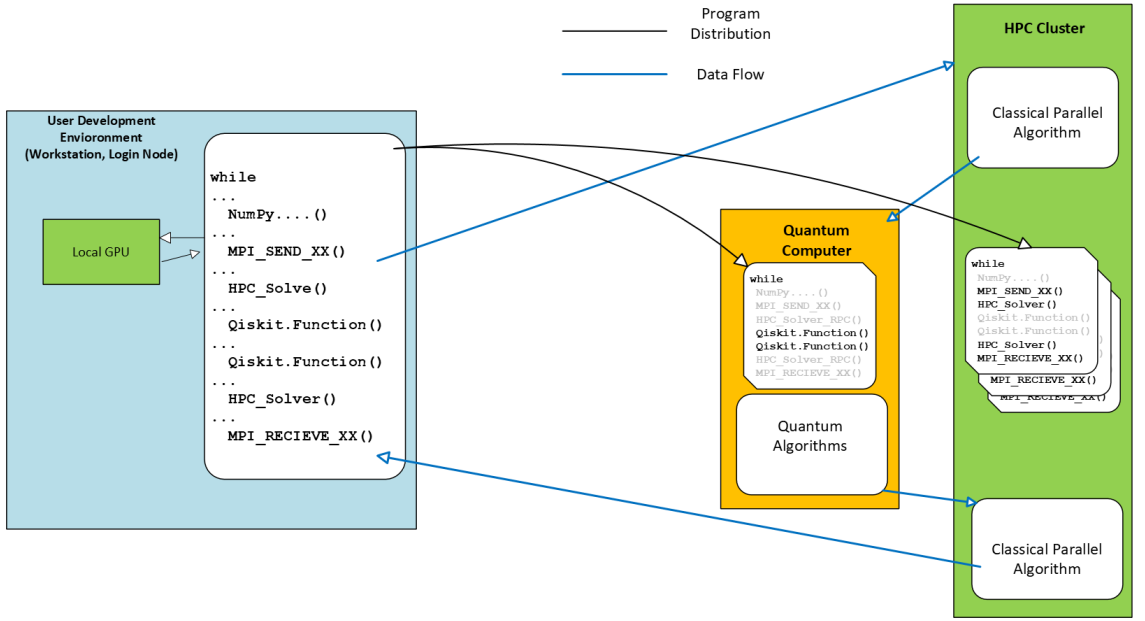
Interactive Python notebook model is excelent for pedagogy, less good for throughput

Stop-and-wait for remote APIs for each phase of a hybrid computation is fundamentally inefficient

Proposal: Distribute the full application control flow to all nodes, expressly or implied in the application binaries.

Using MPI or similar direct communication scheme, feed each step of the pipe directly with the output of the previous one.





Hybrid Binaries for Scalable Execution

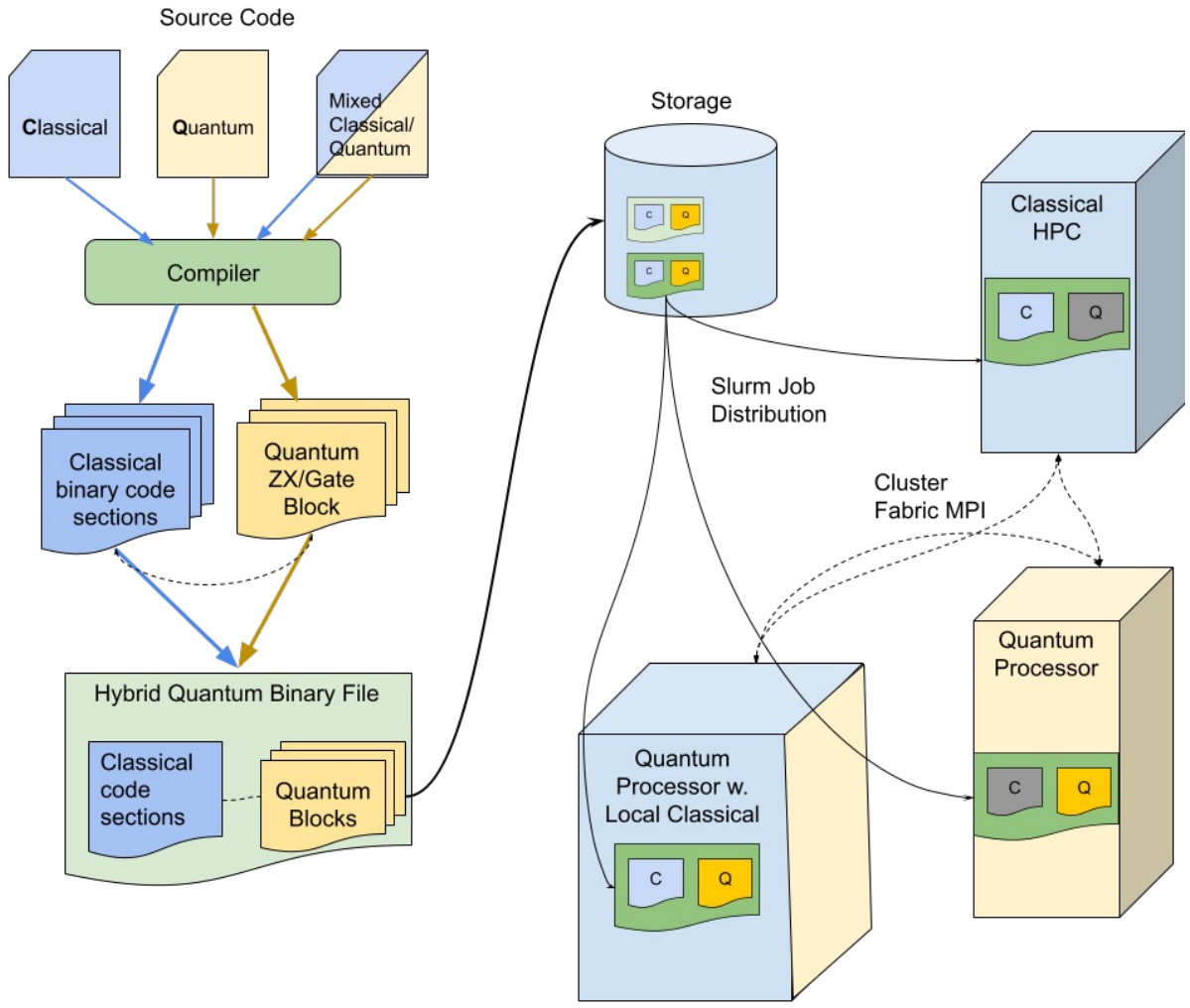
Extending the handling of heterogeneous computing

Compilation of classical and quantum code is very distinct, producing distinct outputs

Hybrid outputs can be wrapped in a single binary file (directly, or by reference).

Control graph is run by all nodes of a parallel hybrid program

Nodes self-identify by capability and run those elements constructed for them.



The Solver Layer

Classical Invocation of Quantum Solvers

Application layer knows which programs need access to quantum resources, but not how they will be used.

Hybrid applications are written in a classical programming language, referencing quantum information only by reference

This is the level at which quantum information lifetimes are managed on quantum resources (qubit registers) organized from the quantum resources allocated to the application.

Quantum registers are passed by reference to quantum solvers, which use quantum primitives and classical logic to provide a solution.

Transformed quantum state from one solver may be passed as an input to the next

```
def find_factor(
    n: int,
    order_finder: Callable[[int, int], int | None] = quantum_order_finder,
    max_attempts: int = 30, ) -> int | None:
    """Returns a non-trivial factor of composite integer n.

    Args:
        n: Integer to factor.
        order_finder: Function for finding the order of elements of the
            multiplicative group of integers modulo n.
        max_attempts: number of random x's to try, also an upper limit
            on the number of order_finder invocations.

    Returns:
        Non-trivial factor of n or None if no such factor was found.
        Factor k of n is trivial if it is 1 or n.
    """
    if sympy.isprime(n):
        print("n is prime!")
        return None

    if n % 2 == 0:
        return 2
    c = find_factor_of_prime_power(n)
    if c is not None:
        return c
    for _ in range(max_attempts):
        x = random.randint(2, n - 1)
        c = math.gcd(x, n)
        if 1 < c < n:
            return c
        r = order_finder(x, n)
        if r is None:
            continue
        if r % 2 != 0:
            continue
        y = x ** (r // 2) % n
        assert 1 < y < n
        c = math.gcd(y - 1, n)
        if 1 < c < n:
            return c

    print(f"Failed to find a non-trivial factor in {max_attempts} attempts.")
    return None
```

Hybrid Binary Composition

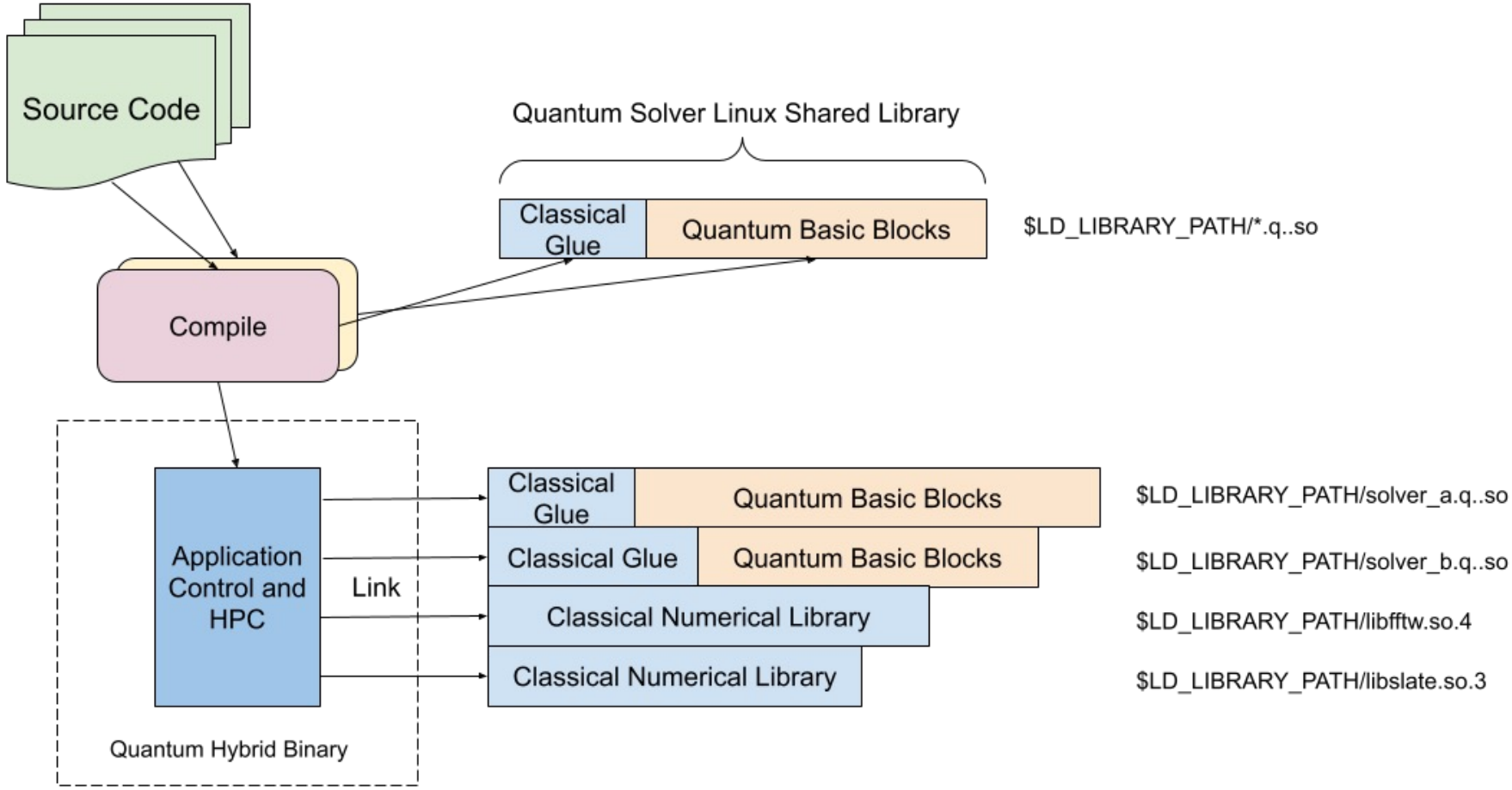
Building solver-based hybrid quantum applications

Imagined as extension to GNU/Linux/ELF, but can be generalized

If Linux model retained, many tools should “just work”.

But an entirely new container system for classical code/data sections and quantum program blocks could be imagined, if necessary

In prototypes, ”Quantum Basic Blocks” may in fact be python modules.



Loading and Executing with Local Quantum

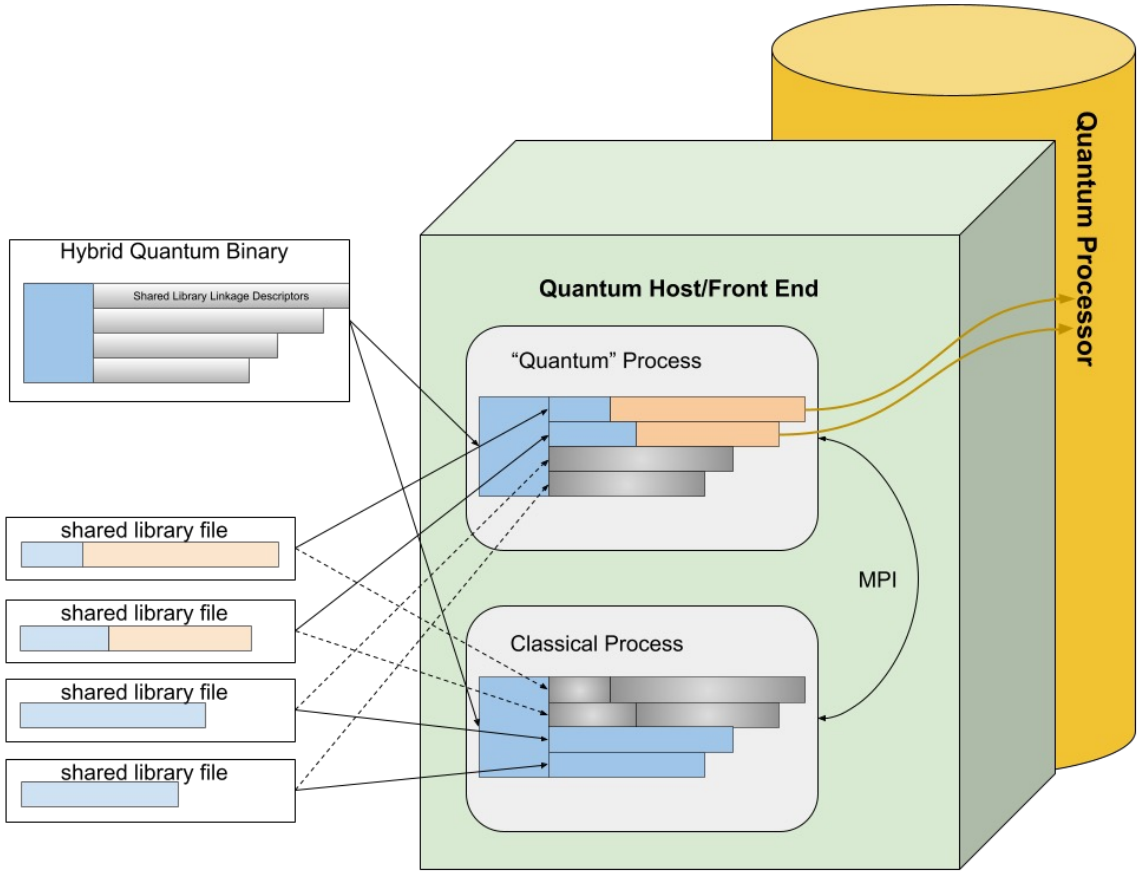
Reasoning about the architecture of hybrid quantum HPC computers

At least one process must be designated to execute quantum blocks, on a node with quantum processing capability.

A process serving program blocks to the device must be non-preemptable.

Not true of a processes doing associated classical pre-post processing.

By splitting processes, resources are no longer locked together.



Separation and Scaling of Classical Components

Reasoning about the architecture of hybrid quantum HPC comput

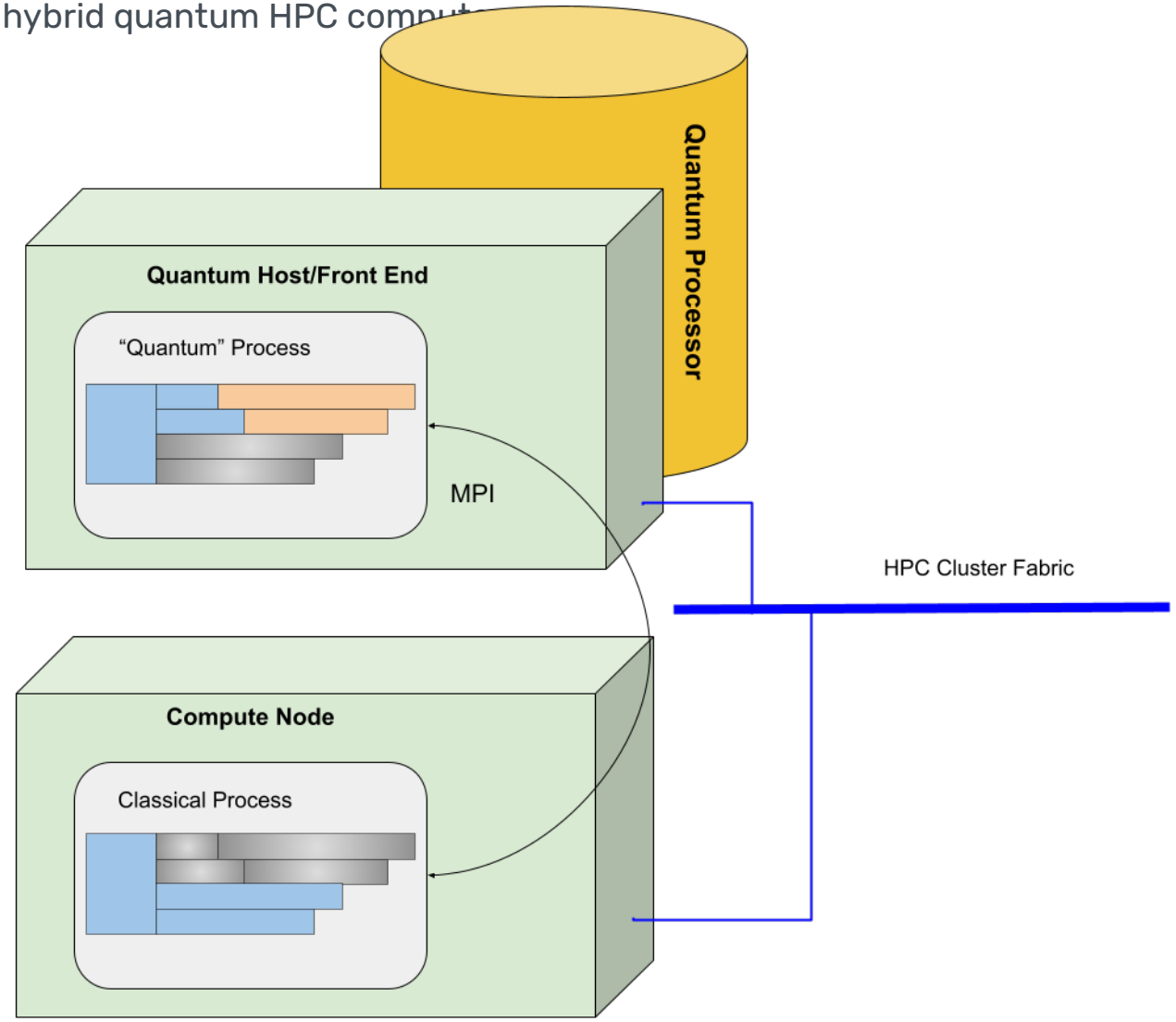
Quantum resource isolation simplfies classical scaling:

Quantum code – quantum basic blocks or QBB – must run on nodes with appropriate support.

Classical components can be run in distinct processes that may or may not be co-located with quantum processing.

Once processes “owning” quantum vs. classical resources are split, classical scaling becomes simple and natural.

Classical control plane must still run on quantum nodes.



Quantum Programming at the Register Level

Hybrid Implementation of Quantum Solvers

Solvers need concise and portable classical signatures, but have possibly complex hybrid implementations.

Reposes on logical qubits, logical gates, classical computation, and classical control.

Users of solvers do not need a deep understanding of quantum algorithms. Solver writers do.

Q# language targeted roughly this level

```
operation QuantumPhaseEstimationImpl (oracle : (Qubit[] => Unit is Adj + Ctl),
                                      targetState : Qubit[],
                                      controlRegister : Qubit[]) : Unit is Adj + Ctl
    {
        body (...)
        {
            let nQubits = Length(controlRegister);
            Microsoft.Quantum.Canon.ApplyToEachCA(H, controlRegister);

            for (idxControlQubit in 0 .. nQubits - 1)
            {
                let control = (controlRegister)[nQubits - 1 - idxControlQubit];
                let power = 2 ^ idxControlQubit;
                Controlled PowerOracle([control], (oracle, targetState, power));

            }

            Adjoint QFTImpl(controlRegister);
        }
    }
```

https://github.com/tsmatz/Quantum-Phase-Estimation-QSharp

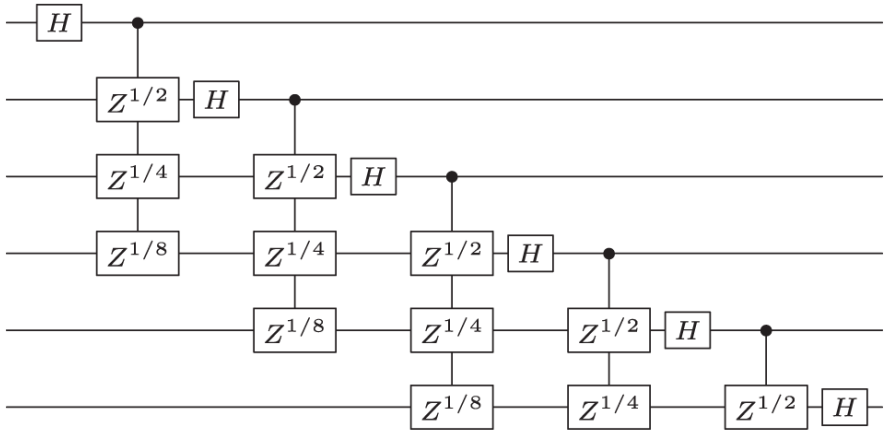
Representing Logical Quantum Computing

Classical Handles for Quantum Information and Operations

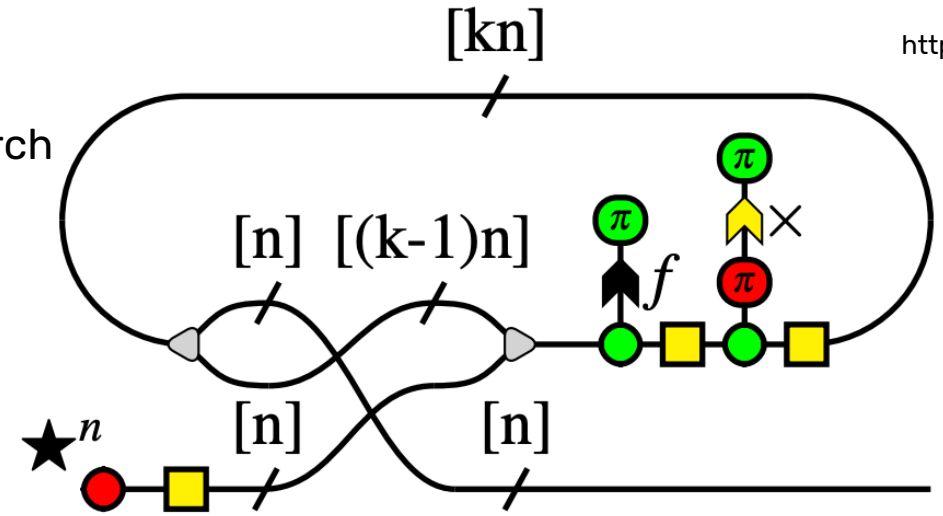
The Logical level provides a definition of a qubit that is modality-independent and idealized.

The logical gate set may be simplified and constrained, but it should be universal.

Human-efficient representations remain a subject of serious research



https://www.nature.com/articles/s41534-020-0257



https://arxiv.org/pdf/2104.01043

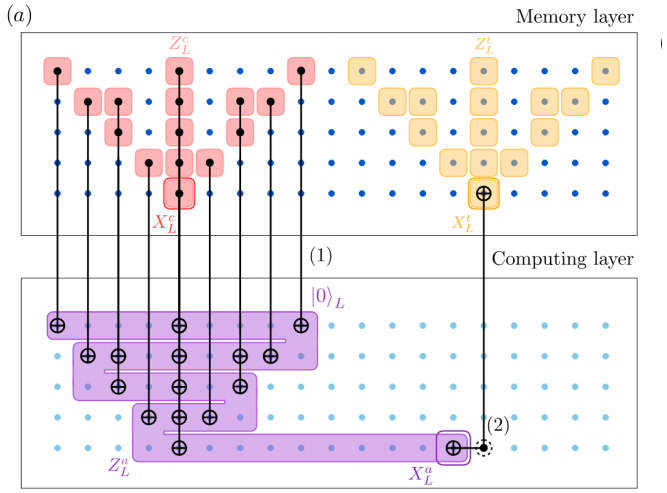
The Virtual Layer

Combining physical quantum and classical state, under classical control

The Virtual layer of abstraction and operation is where Logical qubits and operations are implemented.

Subtly distinct from the underlying physical level
- Uses subset of gates that isn't universal
- Has structured access to measurement sequences, a **temporal dimension**

Stim an example of a language designed for this level




```
# Generated repetition_code circuit.
# task: memory
# rounds: 1000
# distance: 3
# after_clifford_depolarization: 0.001
# Qubit layout: L0 Z1 d2 Z3 d4 Z5 d6
# Legend:
#     d# = data qubit
#     L# = data qubit with logical observable
crossing
#     Z# = measurement qubit
R 0 1 2 3 4 5 6
TICK
CX 0 1 2 3 4 5
DEPOLARIZE2(0.001) 0 1 2 3 4 5
TICK
CX 2 1 4 3 6 5
DEPOLARIZE2(0.001) 2 1 4 3 6 5
TICK
MR 1 3 5
DETECTOR(1, 0) rec[-3]
DETECTOR(3, 0) rec[-2]
DETECTOR(5, 0) rec[-1]
REPEAT 999 {
    TICK
    CX 0 1 2 3 4 5
    DEPOLARIZE2(0.001) 0 1 2 3 4 5
    TICK
    CX 2 1 4 3 6 5
    DEPOLARIZE2(0.001) 2 1 4 3 6 5
    TICK
    MR 1 3 5
    SHIFT_COORDS(0, 1)
    DETECTOR(1, 0) rec[-3] rec[-6]
    DETECTOR(3, 0) rec[-2] rec[-5]
    DETECTOR(5, 0) rec[-1] rec[-4]
}
M 0 2 4 6
DETECTOR(1, 1) rec[-3] rec[-4] rec[-7]
DETECTOR(3, 1) rec[-2] rec[-3] rec[-6]
DETECTOR(5, 1) rec[-1] rec[-2] rec[-5]
OBSERVABLE_INCLUDE(0) rec[-1]
```

https://github.com/quantumlib/Stim/blob/main/doc/file_format_stim_circuit.md

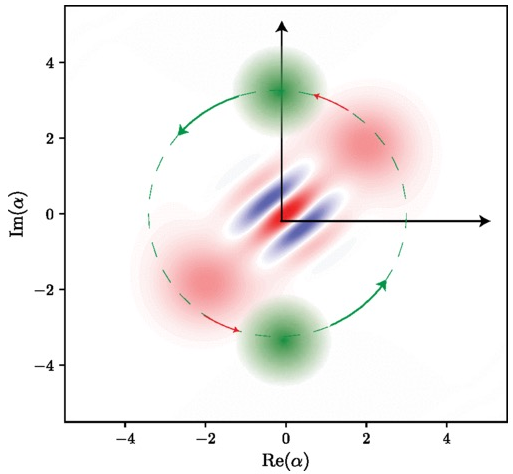
The Physical Layer

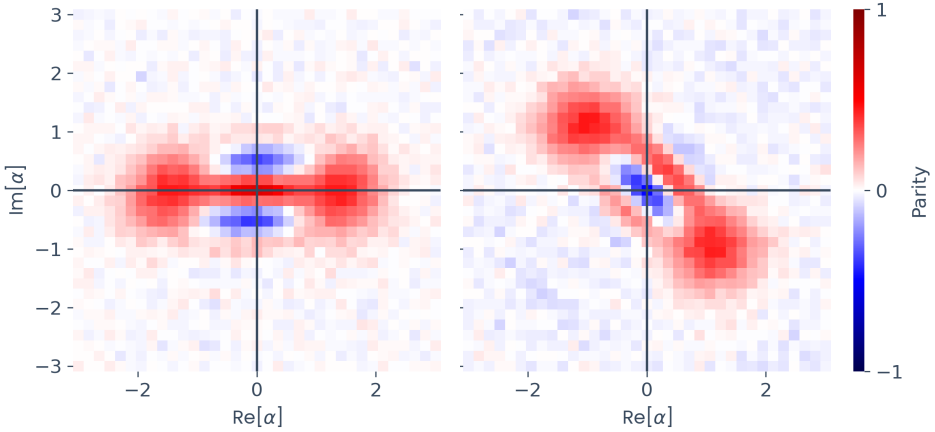
Extracting computation from a controlled quantum device

Possible/practical gate sets determined by quantum modality and control scheme.

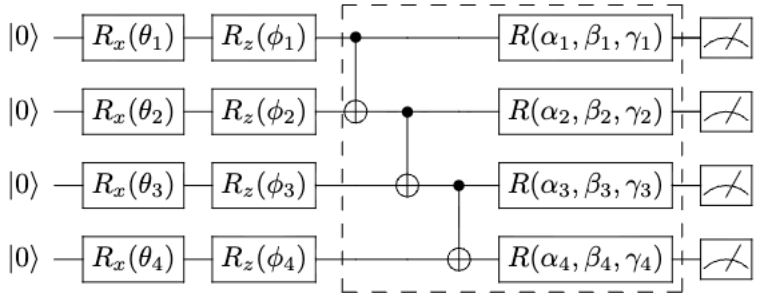
Logical gate set must be implementable by physical gate set.

But they need not be the same gate set.





https://journals.aps.org/prx/abstract/10.1103/PhysRevX.9.041053



https://arxiv.org/abs/1907.00397

Axis of Exploration: Temporal Decomposition

There are trade-offs to examine for when and where abstractions are realized

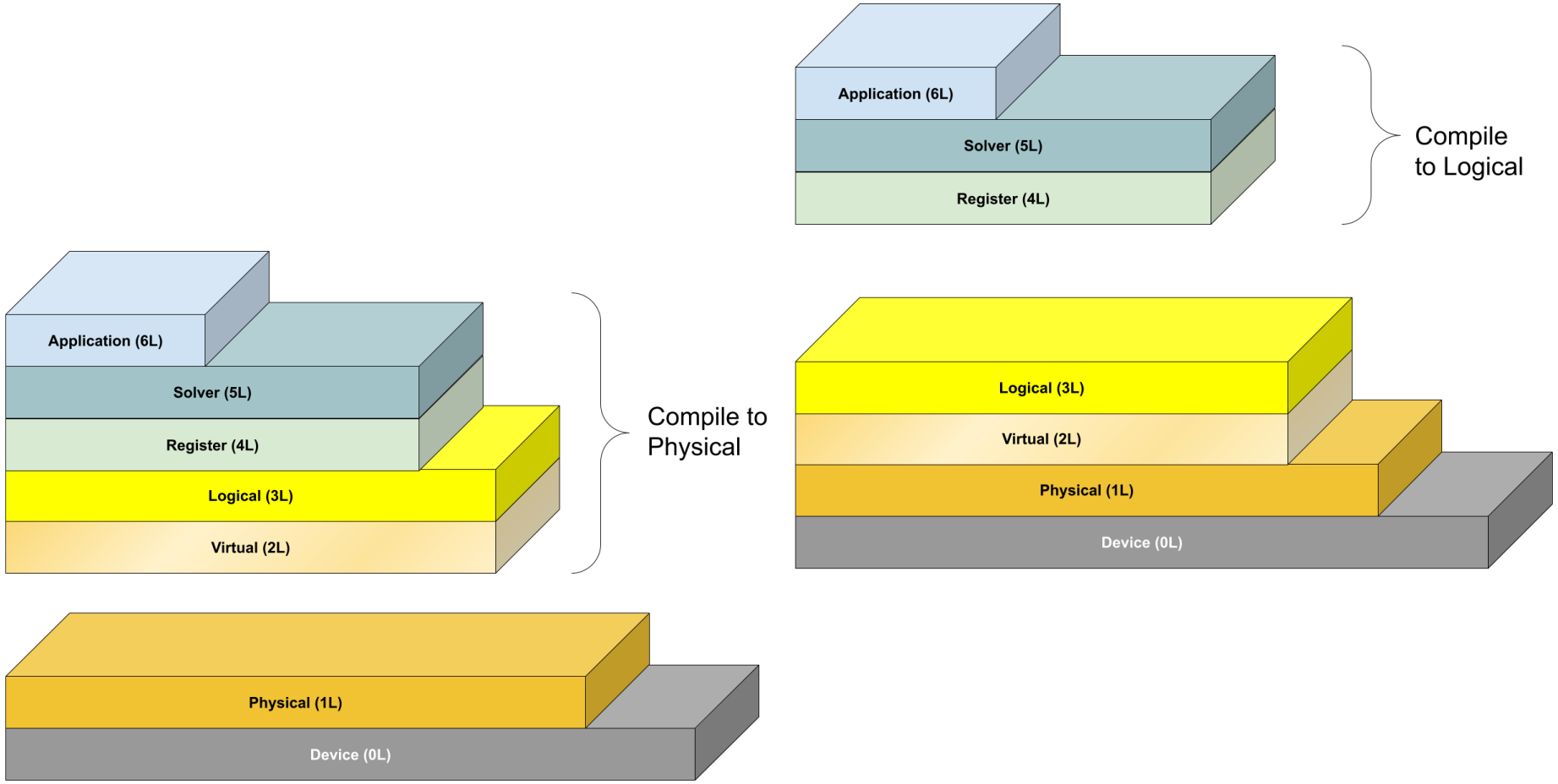
Compile-time vs. Run-time

Compile-to-physical generates classical control and error-correction code using logical/physical mappings determined in advance.

Compile-to-logical generates logical operation descriptions, defers virtualization to run-time.

Minimum run-time work vs. greater run-time flexibility

Physical as well as Logical quantum basic blocks?

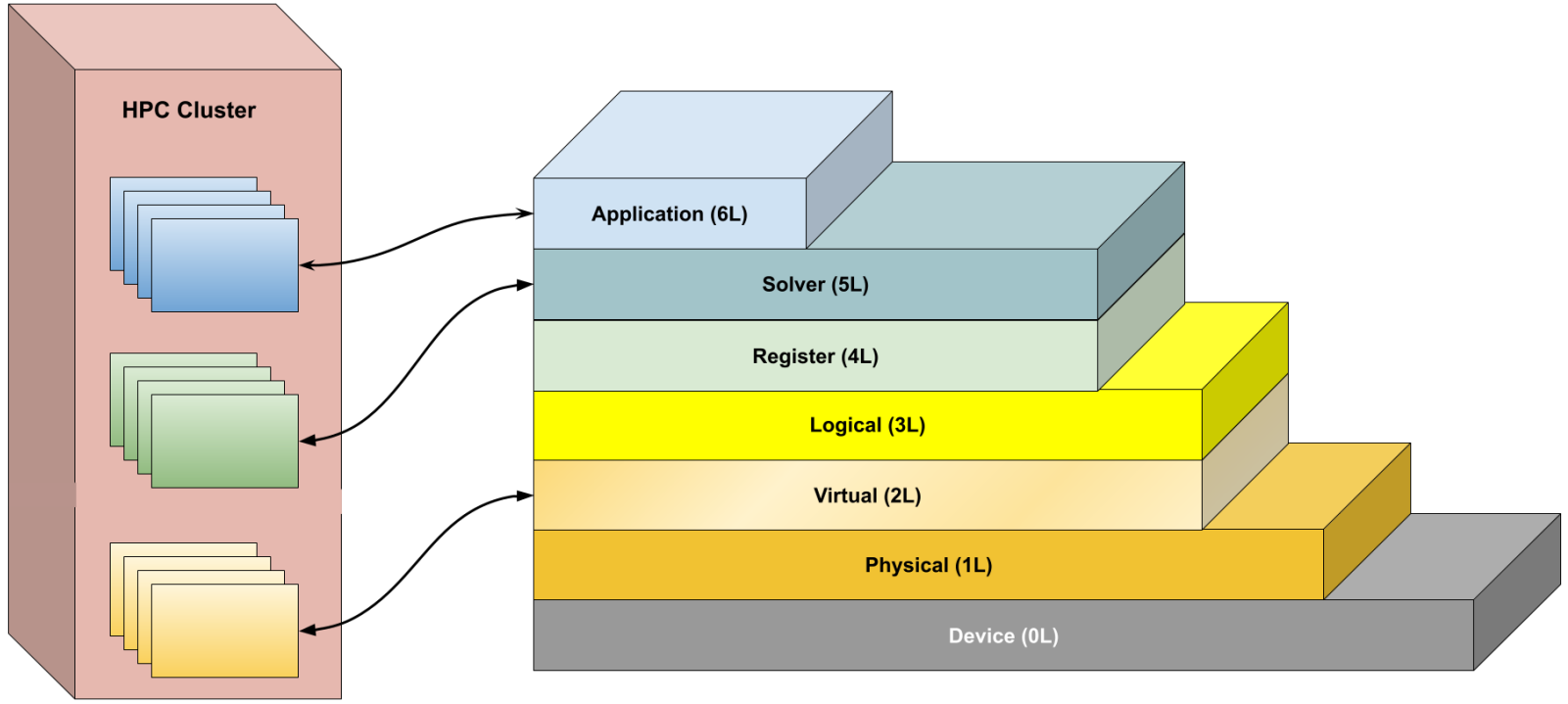


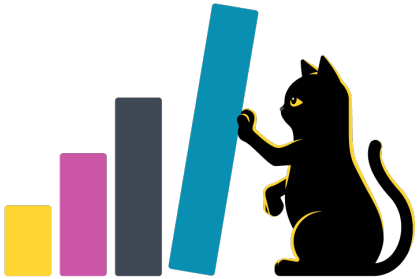
Separation of Concerns, Independence of Acceleration

Reasoning about the architecture of hybrid quantum HPC computers

HPC classical computation can play a role at various levels of abstraction

If properly architected, it's invisible to layers above and below whether a given layer is implemented in part with parallel classical computing





Mapping and Navigating the Ecosystem

Filling gaps and finding friends

- Alice & Bob do not intend to independently develop a full toolchain and stack
- Ideas presented here are a way of looking at things that perhaps suggests better ways of doing things
- Looking for partners and collaborators with commonality of vision
- White paper in the works, but happy to have follow-up conversations

THANK YOU.

ALICE & BOB